

# Quality Minded — First Meetup



# Welcome, Quality Minders!

---

Hi, I'm Katja.

Zoya and I created this community to talk about one big question:

**How do we keep quality at the center in the age of AI?**

Between the two of us, we have about **40 years of QA experience** — so we've survived a few “big changes” already.

Now all the QAs are facing another one — with plenty of questions, opportunities, and probably a few new bugs waiting for us.

**We invite you to go through this change together and figure out the future of QA along the way.**

Welcome to our first meetup!

# Why the QA function needs to change

---

- Agentic development is dramatically faster than traditional development.
- The cost of fixing small bugs is now much lower — the economics of testing have changed.
- Bugs from AI-generated code are structurally different from bugs human developers produce.

# How bugs are changing

---

The shape of defects is shifting. Two recent studies put numbers on it — one from industry review data, one from academic static analysis at scale. Both point the same direction: AI-authored code doesn't just contain more issues, it contains different issues, concentrated in categories that used to be rare.

Sources:

- CodeRabbit — State of AI vs. Human Code Generation Report (December 17, 2025)

<https://coderabbit.ai/content/assets/code-rabbit-state-of-ai-vs-human-code-generation-report-lite.pdf>

- Cotroneo, Improta, Liguori (Univ. of Naples Federico II, 2025) — Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity

<https://arxiv.org/pdf/2508.21634>

Next two slides: what each one actually found.

# CodeRabbit: what AI PRs actually look like

---

470 real GitHub PRs reviewed by CodeRabbit — 320 AI co-authored, 150 human-only. Normalized to issues per 100 PRs.

Volume:

- ~1.7× more issues overall (10.83 vs. 6.45 per PR).
- Heavier tail: 95th percentile — AI 39 issues vs. humans 23.

Where the gap is worst:

- Logic & correctness — 1.75×. Algorithm/business-logic errors alone 2.25×.
- Error & exception handling — ~2×.
- Concurrency — 2.29×. Null/None dereference — 2.27×.
- Security — 1.57×. Password handling 1.88×, XSS 2.74×.
- Excessive I/O — ~8×. Rare overall, but almost entirely an AI problem.

Where humans still lose:

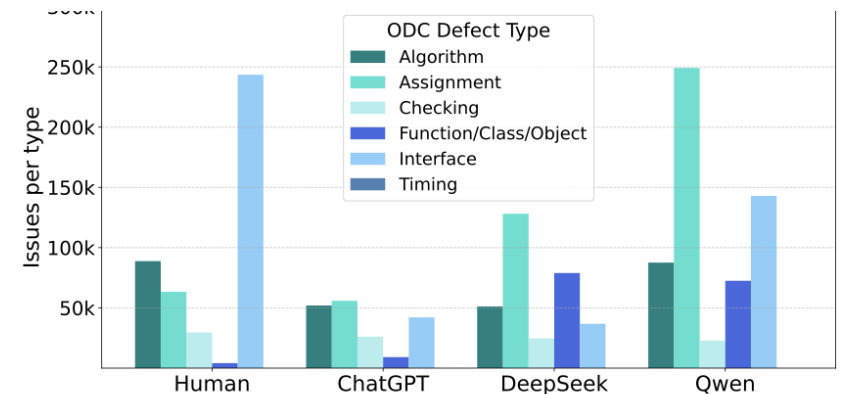
- Spelling 1.76× worse
- Testability issues also appear more often in human code (23.65 vs. 17.85)

# Naples study: defect profiles diverge

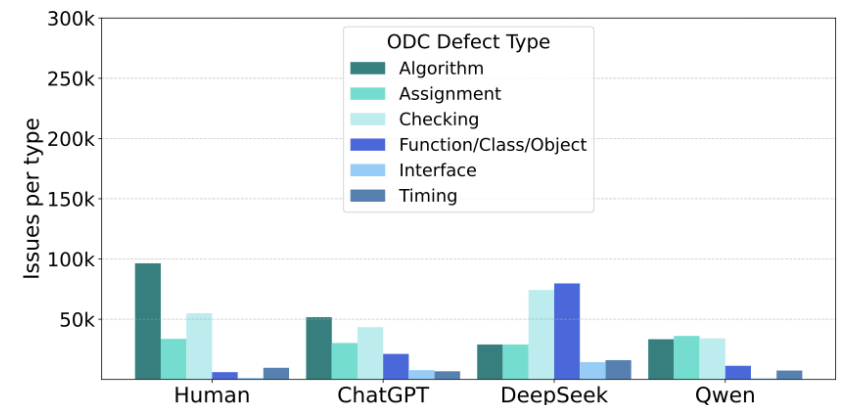
Cotroneo et al., 2025 — >500k Python + Java samples. Human code from 17k GitHub repos vs. ChatGPT, DeepSeek-Coder, Qwen-Coder.

Defect profiles are structurally different

- Human code: interface and algorithm defects — complex control flow, cyclomatic complexity, exception handling. Signature of mature, maintained codebases.
- AI code: assignment and function/class/object defects — unused arguments, unused variables, redefined names, too-few-public-methods. Syntactically valid but semantically shallow.
- Language matters: AI is better than humans at Python (−4k defective samples avg), much worse at Java (+22k).



(a) Python



# And it keeps changing

---

The defect profile **isn't stable**. It moves as the models move.

The bugs Claude was producing a year ago aren't the bugs the latest models produce today. Different failure modes, different blind spots, different categories of "looks right, isn't."

Which means: whatever heuristics we build this quarter will be partly wrong next quarter. This isn't a fixed target we're learning to hit — it's a moving one. The only durable advantage is a community that keeps comparing notes in real time.

# What we suggest

---

- **Share experience.**

I'd like us to meet online at least every 2 weeks. Speakers welcome — bring what you're seeing. What broke, what worked, what surprised you. That's the whole point.

- **Try different approaches.**

There are quality minders from different countries and companies here. We don't need to converge on one method — we need to try many, and compare results honestly. That's how the picture gets clearer, faster than any of us could alone.

# What I'm trying right now

---

Standard test design produces a lot of test cases.

On one project I had an automated suite of ~70 cases catching 3–4 bugs per release — exploratory testing caught the other ~20.

So now I'm experimenting with two ideas:

- "Behave as a first-time user" agent — automation that doesn't know the happy path, doesn't follow the script, and does what a real new user would actually do.
- Client-demo agent — automation that walks through the product the way you'd show it to a prospect. If it can't get through a demo cleanly, neither can you.

Both are early.

# How to try

---

The last section of the today meeting is about the open-source platform we're using internally.

It's early stage — but feel free to use it, break it, and tell us what happened. Share your ideas, your results, and the agents that actually work for you.

I'll also share the two agents I mentioned — the first-time user and the client demo — so you can experiment with them directly. I genuinely want to hear whether they work for you, and what works better.

# Questions?

---

Ask now or later in our group <https://www.meetup.com/quality-minded>

Contact: [ekaterina.tuna.v@gmail.com](mailto:ekaterina.tuna.v@gmail.com)